

Store Backends

Simon Frost

Overview

RDFLib.jl supports multiple storage backends for RDF graphs. This vignette covers the built-in stores: in-memory, SQLite, DuckDB, auditable, concurrent, and batch.

```
using RDFLib
ex = Namespace("http://example.org/")
```

```
Namespace("http://example.org/")
```

MemoryStore (Default)

The default in-memory store is fast and suitable for most use cases.

```
# MemoryStore is the default
g = RDFGraph()
println("Store type: ", typeof(g.store))

add!(g, Triple(ex("a"), ex("p"), Literal("hello")))
add!(g, Triple(ex("b"), ex("p"), Literal("world")))
println("Triples: ", length(g))
```

```
Store type: MemoryStore
Triples: 2
```

SQLite Store

Persist your graph to a SQLite database for durability and larger-than-memory graphs.

```

# Create a SQLite-backed graph
db_path = tempname() * ".db"
store = SQLiteStore(db_path)
g = RDFGraph(; store=store)

# Add data
for i in 1:100
    add!(g, Triple(ex("item$i"), RDF.type, ex("Item")))
    add!(g, Triple(ex("item$i"), ex("value"), Literal(i)))
end

println("SQLite graph: $(length(g)) triples")

# Query works the same way
results = collect(triples(g, (nothing, RDF.type, ex("Item"))))
println("Items found: $(length(results))")

```

SQLite graph: 200 triples
Items found: 100

```

# Data persists - reopen the same file
store2 = SQLiteStore(db_path)
g2 = RDFGraph(; store=store2)
println("Reopened: $(length(g2)) triples")

# Clean up
rm(db_path; force=true)

```

Reopened: 200 triples

DuckDB Store

DuckDB provides excellent analytical query performance for large graphs.

```

db_path = tempname() * ".duckdb"
store = DuckDBStore(db_path)
g = RDFGraph(; store=store)

# Bulk insert
for i in 1:50

```

```

    add!(g, Triple(ex("sensor$i"), RDF.type, ex("Sensor")))
    add!(g, Triple(ex("sensor$i"), ex("reading"), Literal(rand() * 100)))
    add!(g, Triple(ex("sensor$i"), ex("location"), Literal("Zone $(i % 5 + 1)")))
end

println("DuckDB graph: $(length(g)) triples")

# Clean up
rm(db_path; force=true)

```

DuckDB graph: 150 triples

Auditable Store

Track every change with undo/redo support:

```

base_store = MemoryStore()
store = AuditableStore(base_store)
g = RDFGraph(; store=store)

# Make some changes
add!(g, Triple(ex("a"), ex("p"), Literal("original")))
println("After add: $(length(g)) triples")

add!(g, Triple(ex("b"), ex("q"), Literal("second")))
println("After second add: $(length(g)) triples")

# Undo the last change
undo!(store)
println("After undo: $(length(g)) triples")

# Check what's left
for t in collect(triples(g))
    println("  ", t)
end

```

After add: 1 triples

After second add: 2 triples

After undo: 1 triples

```
(URIRef("http://example.org/a"), URIRef("http://example.org/p"), Literal("original"))
```

Concurrent Store

Thread-safe wrapper for multi-threaded applications:

```
base = MemoryStore()
store = ConcurrentStore(base)
g = RDFGraph(; store=store)

# Safe to use from multiple threads
add!(g, Triple(ex("x"), ex("y"), Literal("z")))
println("Concurrent store: $(length(g)) triples")
```

Concurrent store: 1 triples

Batch Add Graph

Optimize bulk insertions by batching:

```
g = RDFGraph()

# Use BatchAddGraph for efficient bulk loading
batch = BatchAddGraph(g)

for i in 1:1000
    add!(batch, Triple(ex("item$i"), RDF.type, ex("Item")))
end

# Flush to commit all at once
flush!(batch)
println("Batch inserted: $(length(g)) triples")
```

Batch inserted: 1000 triples

Store Comparison

Store	Persistent	Speed	Thread-Safe	Best For
MemoryStore		Fast		Small-medium graphs, prototyping

Store	Persistent	Speed	Thread-Safe	Best For
SQLiteStore		Medium		Persistent storage, medium graphs
DuckDBStore		Medium		Analytics, large graphs
ConcurrentStore	depends	Overhead		Multi-threaded apps
AuditableStore	depends	Overhead		Change tracking, undo/redo
BatchAddGraph	depends	Bulk		Data loading

Transactions

Some stores support transactions for atomic operations:

```
store = SQLiteStore()
g = RDFGraph(; store=store)

# Transaction ensures all-or-nothing
transaction(store) do
    add!(g, Triple(ex("a"), ex("p"), Literal("1")))
    add!(g, Triple(ex("b"), ex("q"), Literal("2")))
end

println("After transaction: $(length(g)) triples")
```

After transaction: 2 triples